

Kodin Code Session Report - Parneet

Date: 25/9/2026, 3:17:46 pm

QUESTION

Create a Java program using an abstract class Vehicle that contains an abstract method startEngine(). Create two subclasses, Car and Motorcycle, that implement the startEngine() method.

The program should:

Create objects of Car and Motorcycle.

Call the startEngine() method for both vehicles.

Display different messages for starting the car and motorcycle.

AIM

Write a program and attach UML diagram using an abstract class Vehicle that contains an abstract method startEngine(). Create two subclasses, Car and Motorcycle, that implement the startEngine() method.

The program should include:

Create objects of Car and Motorcycle

Call the startEngine() method for both Vehicles

Display the different messages for starting the car and Motorcycle

ALGORITHM

Step 1: Start
Step 2: Define an Abstract class Vehicle
Step 3: Declare an abstract method startEngine() without a body
Step 4: Define subclass Car
Step 5: Extend the abstract class Vehicle
Step 6: Override the startEngine() method to display a message specific to starting a car.
Step 7: Define subclass Motorcycle
Step 8: Extend the abstract class Vehicle
Step 9: Override the startEngine() method to display a message specific to starting a motorcycle.
Step 10: Create main class
Step 11: Instantiate an object of the Car class
Step 12: Instantiate an object of the Motorcycle class
Step 13: Call the startEngine() method using the Car object.
Step 14: Call the startEngine() method using the Motorcycle object.
Step 15: Stop

SOLUTIONS

File: code.java

```
abstract class Vehicle
{
    abstract void startEngine();
}
class Car extends Vehicle
{
    @Override
    void startEngine()
    {
        System.out.println("Car Engine Started");
    }
}
class Motorcycle extends Vehicle
```

```

{
    @Override
    void startEngine()
    {
        System.out.println("Motorcycle Engine Started");
    }
}
public class code
{
    public static void main (String args[])
    {
        Car c = new Car();
        Motorcycle m = new Motorcycle();
        c.startEngine();
        m.startEngine();
    }
}

```

PROGRAM OUTPUT

```

===== code.java =====
Car Engine Started
Motorcycle Engine Started

```

CONCLUSION

The program was executed successfully while implementing of abstraction and polymorphism

External Activity / Applications Found During Session

No external applications, window switches, or copy/paste events were recorded during this session.